Assignment 1: Reinforcement Learning

Name: Vincent Yang
GitHub username: vi-yang-NU
Repository name: RL
Link to the GitHub repository: https://github.com/vi-yang-NU/RL

Instructions for running the code

> Instructions of how to run your code.

These instructions are for grading. Every answer, plot, evaluation statistic and validation result is in the report, and the repository includes the results the report was built from (`results/`), so **nothing has to be run to grade**. The steps below set up the code to rerun any part, and open a dashboard that shows each part's answers next to its plots and test evidence.

1. Quick start

You need Python 3.9 or newer on macOS or Linux (on Windows, use WSL). In a terminal:

```
git clone https://github.com/vi-yang-NU/RL.git
cd RL
./setup.sh --no-ssh           # creates .venv, installs the packages, checks the environment
source .venv/bin/activate
python run.py ui              # opens the dashboard in your browser at http://localhost:8501
```

`setup.sh` installs the packages in `requirements.txt` (NumPy, Matplotlib, PyTorch, Gymnasium with the Atari environments, ale-py) and `requirements-ui.txt` (Streamlit, pandas, reportlab) into a virtual environment in `.venv`; the first time takes a few minutes because of PyTorch. It ends with `python run.py check`, whose last line should read `ready`. `--no-ssh` skips the optional SSH setup for Northwestern's deepdish servers. Stop the dashboard with Ctrl+C in the terminal.

Without the script:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt -r requirements-ui.txt
python run.py check
```

2. Where to find each answer

The report and the dashboard follow the handout's order and use the same headings. In the dashboard, open the tab, then scroll to the section; in the report, the same section is under the part's heading.

Handout item	Dashboard tab › section	Rerun with
GitHub username, repository and link; these instructions	Overview	

Handout item	Dashboard tab › section	Rerun with
2: MDP formulation and assumptions	2 · Value & Policy Iteration › MDP formulation, Assumptions	
2 (a) Enumeration, T = 500: optimal value function at time 0	2 · Value & Policy Iteration › (a) Enumeration, T = 500	`python run.py dp --method enumeration`
2 (b) Value iteration: optimal value function	2 · Value & Policy Iteration › (b) Value iteration, T = ∞	`python run.py dp --method value_iteration`
2 (c) Policy iteration: optimal policy	2 · Value & Policy Iteration › (c) Policy iteration	`python run.py dp --method policy_iteration`
2: edge cases, validation tests, 3 more months	2 · Value & Policy Iteration › Edge cases, Tests for validating..., Additional validation...	`python run.py tests --section dp`
Part 1 (a) CartPole-v1: training curve, histogram, mean and standard deviation	3 · Policy Gradient Part 1 › (a) CartPole-v1	`python run.py pg --part 1 --env cartpole`
Part 1 (b) Pong-v5: the same	3 · Policy Gradient Part 1 › (b) Pong-v5	`python run.py pg --part 1 --env pong`
Part 1: edge cases, validation tests, 3 more months	3 · Policy Gradient Part 1 › Edge cases, Tests for validating..., Additional validation...	`python run.py tests --section pg1`
Part 2: the baseline chosen and how it is implemented	Policy Gradient Part 2 › Baseline: choice and implementation	
Part 2 (a) CartPole-v1 with the baseline	Policy Gradient Part 2 › (a) CartPole-v1 with baseline	`python run.py pg --part 2 --env cartpole`
Part 2 (b) Pong-v5 with the baseline	Policy Gradient Part 2 › (b) Pong-v5 with baseline	`python run.py pg --part 2 --env pong`
Part 2: edge cases, validation tests, 3 more months	Policy Gradient Part 2 › Edge cases, Tests for validating..., Additional validation...	`python run.py tests --section pg2`

The files behind them are in `results/` (section 8 lists the folders).

3. Using the dashboard

`python run.py ui` opens the dashboard at http://localhost:8501.

- **Tabs** along the top:
  - **Overview**: the submission details, a table of where each part is, progress, and the report and these instructions as PDFs to download.
  - **2 · Value & Policy Iteration**, **3 · Policy Gradient Part 1** and **Policy Gradient Part 2**: one tab per part of the handout.
- **Contents**, at the right edge of the page, lists the sections of the open tab. The entry for the section on screen is highlighted, and the highlight glides to the next entry as you scroll; click an entry to jump to it. (It appears when the window is at least about 1250 px wide.)
- **A part's tab** lists the handout's questions in order, each in its own box: the question quoted from the handout, the required plots, for policy-gradient runs the mean and

standard deviation over the 500 evaluation episodes, the validation tests, and the **Answer**. Click a validation test to expand it: what it does, how it validates the algorithm, each check (✓ or ✗), and its metrics, tables and figures.

- **Explore the problem** (tab 2, under the Run panel) is an interactive view of Section 2. Change any constant or structural switch of the shuttle problem on the left (or pick a preset such as deterministic arrivals, γ = 0, a tiny station or two shuttles per period) and the three solvers rerun immediately. The page on the right shows the two stations with the waiting and delivered customers; **Step**, **Play** and **Back** replay the first 10 periods of the chosen policy, the action buttons take a different action in a period, and the tiles and the period table show the state, the cost and its breakdown, V*(s_t), Q*(s_t, ·) and the running discounted total at each step. The panel beside it steps through the first 10 iterations of enumeration, value iteration or policy iteration (values at selected states, the dispatch threshold, the change per step), and the charts show V*, the policy bands and the convergence for the selected step. **Run this case in the Run panel** (under the widgets) copies the current parameters into the Run panel at the top of the tab, as `--set` arguments with a run tag so the handout's results stay, and scrolls up to it; press **Launch** there.
- **The Run panel** at the top of each part's tab reruns that part; click its header to collapse it. Choose a job (for example *(a) CartPole-v1* or *Validation tests*), optionally add a run tag and extra arguments, and press **Launch**. Under **Jobs**, choose a job to see its log and, while a policy-gradient run trains, its reward curve (turn on auto-refresh to follow it). When the job has finished, press R to reload the page and its plots.
- **The sidebar** on the left sets where jobs run: **this computer** (the default) or a deepdish server, and the GPU. Hide it with « at its top; bring it back with » at the top left of the page. The deepdish connection settings and the **Set up deepdish** panel in it are only needed to run on deepdish.

Rerunning a part overwrites its results in your copy of the repository; `git checkout results/` restores the submitted ones. For a policy-gradient run, a **Run tag** (or `--tag NAME` on the command line) writes to a separate folder instead and leaves the submitted run in place.

If the dashboard does not load

Nothing in the dashboard is needed to grade: the report (`report/report.pdf`) has every answer, plot, statistic and test result, and the files behind them are in `results/` (section 8). If the page stays blank, keeps saying "Please wait" or shows a connection error:

1. Wait for the terminal to print `Dashboard: http://localhost:8501`: the first start takes up to a minute because of PyTorch. Then reload the page (press R in the page, or the browser's reload button).
2. If no browser opened, or it opened before the dashboard was ready, open http://localhost:8501 yourself. Over SSH the dashboard does not open a browser: forward the port (`ssh -L 8501:localhost:8501 …`) and open that address on your computer.
3. If the port is in use (the terminal says so), start it elsewhere: `python run.py ui --port 8502`.
4. If `python run.py ui` fails to start, install the dashboard's packages with `pip install -r requirements-ui.txt` (with `.venv` active), or run Streamlit directly: `python -m streamlit`

`run ui/app.py`.

5. Try another browser or a private window: some extensions and strict ad blockers block the page's WebSocket connection, which is what the dashboard uses to draw.

If it still does not load, grade from the report: section 2 names the report section for each handout item, section 4 reruns any part from the command line, and `python run.py report` rebuilds the report from `results/`.

4. Rerunning from the command line

Every part runs through one entry point, `run.py`, from the repository root with the virtual environment active (`source .venv/bin/activate`); `python run.py -h` lists the commands.

Part	Command	Results in
Section 2 (a)–(c): enumeration, value and policy iteration	`python run.py dp`	`results/dp/`
Part 1 (a): CartPole-v1	`python run.py pg --part 1 --env cartpole`	`results/pg/part1_cartpole/`
Part 1 (b): Pong-v5	`python run.py pg --part 1 --env pong`	`results/pg/part1_pong/`
Part 2 (a): CartPole-v1 with a baseline	`python run.py pg --part 2 --env cartpole`	`results/pg/part2_cartpole/`
Part 2 (b): Pong-v5 with a baseline	`python run.py pg --part 2 --env pong`	`results/pg/part2_pong/`

Section 2. `python run.py dp` solves the shuttle-dispatch MDP (K = 15, c_f = 100, c_h = 2, A_t ~ Unif{1, ..., 5}, at most 200 customers waiting, γ = 0.95) with all three methods; `--method enumeration`, `--method value_iteration` or `--method policy_iteration` runs one of them. Each method saves its solution as JSON and its plot: `enumeration_value.png`, `value_iteration_value.png` and `policy_iteration_policy.png`, plus a convergence plot. Any constant of the problem can be changed with `--set`, for example `python run.py dp --set capacity=10 --set gamma=0.9 --set 'arrivals=[1,2,3]'` (`rl/dp/shuttle.py` lists the fields, including the structural switches `max_dispatch` and `holding_charged`). `--backend cuda` runs the same three methods with the CUDA kernels in `rl/cuda/dp.py` (one thread per state, the value function in shared memory, the convergence loop inside the kernel); on a machine without an NVIDIA GPU they run in Numba's CPU simulator, which is slow but gives the same numbers. `--explorer results/dp/explore.html` also writes the dashboard's interactive explorer (section 3) for the chosen parameters as a standalone page that opens in any browser.

Policy gradient. Each run trains the policy network with Adam, then rolls out 500 episodes with the trained policy. CartPole-v1 trains on a CPU; Pong-v5 is much slower and is best run on a GPU (section 6). A run's folder contains:

- `training_curve.png`: episode reward against training episode, with the 100-episode moving average;
- `eval_histogram.png`: histogram of the 500 evaluation rewards;

- `eval.json`: the 500 evaluation rewards, their mean and their standard deviation;
- `config.json` (the settings used) and `train.json`.

The defaults follow the handout: learning rate 0.01, discount factor 0.95 for CartPole and 0.99 for Pong, and 500 evaluation episodes. An episode ends when the environment ends it (CartPole-v1 stops itself after 500 steps; a Pong game ends at 21 points) or after an upper limit of 1000 steps (`--set max_steps=N`; for Pong the limit is 10000, because a game lasts 1000-3000 steps and a 1000-step cap would cut games short). Options:

Option	Effect
`--episodes N`	number of training episodes (default 1000 for CartPole, 2000 for Pong)
`--lr X`, `--gamma X`	Adam learning rate and discount factor
`--baseline NAME`	Part 2 baseline: `value` (learned state-value network, the default), `mean` (constant moving average) or `time` (per time step)
`--seed N`	random seed (default 0)
`--device D`	`auto` (the GPU when one is visible, else the CPU), `cpu`, `cuda`, `cuda:N` or `mps` (Apple GPU); `cuda` and `mps` fail if unavailable. The policy, the baseline network and every tensor they use are placed on that device; the run prints the GPU it uses
`--eval-episodes N`	number of evaluation episodes (default 500)
`--tag NAME`	save to `results/pg/<run>_NAME/`, next to the submitted run
`--eval-only`	skip training and evaluate the run's saved `policy.pt`
`--set KEY=VALUE`	any tunable in `rl/pg/options.py` (repeatable): network sizes (`'hidden=[128,64]'`), `arch=cnn` for Pong, `batch_episodes`, `normalize`, `reward_to_go`, `discount_grad`, `entropy_coef`, `max_grad_norm`, `eval_greedy`, the baseline's learning rate, `kernels=true` (discounted returns and Pong preprocessing with the CUDA kernels in `rl/cuda/pg.py`), ...

5. Validation experiments

The experiments that check that each algorithm behaves as predicted are in `tests/test_dp.py`, `tests/test_pg_part1.py` and `tests/test_pg_part2.py`. Each one states what it does and how it validates the algorithm; the report's "Tests for validating the correctness of your algorithm" sections list them with their results.

```
python run.py tests --list                # every experiment, by section
python run.py tests --section dp          # dp, pg1, pg2 or all
python run.py tests --section pg1 -k TEXT # only experiments whose name or title contains TEXT
```

Each experiment ends as **passed** (all its checks hold), **failed** (a check does not hold), **error** (it raised an exception) or **recorded** (evidence without pass/fail checks). Results go to `results/tests/<section>.json`, with figures in `results/tests/<section>/`. All of them run on a laptop CPU in about two minutes.

Unit tests. `python run.py unittest` runs about 110 unit tests (`tests/unit/`, standard-library `unittest`) of the building blocks and edge cases: MDP construction, the

solvers on problems with closed-form solutions, exact-arithmetic references, discounted returns, policy networks, environment adapters, training on toy environments, the baselines, device placement, and the CUDA kernels (compared with the CPU code; on a machine without an NVIDIA GPU they run in Numba's simulator, which adds about a minute).

Lean proofs. `lean/` contains Lean 4 (Mathlib) proofs of the properties the tests check against: the Bellman operator of the shuttle MDP is a γ-contraction and the value-iteration error bound is sound, the optimal value is monotone in the horizon and in the queue, policy improvement never makes a policy worse, the backward recursion for discounted returns equals its definition, and an action-independent baseline leaves the expected policy gradient unchanged. The exact reference values computed by the verified Lean definition are committed in `lean/reference/shuttle_reference.json` and compared with the Python solvers by the tests, so Lean does not have to be installed to grade; `lean/README.md` explains how to rebuild the proofs (`lake build`) if you wish.

6. Running on deepdish (one GPU card)

The handout requires a single GPU card on deepdish. Clone the repository there, run `./setup.sh --server` (it detects the servers' GTX 1080 cards and installs a PyTorch build that supports them, and puts the environment on the local disk because importing PyTorch from the NFS home takes minutes), check which cards are free with `nvidia-smi`, and choose one with `CUDA_VISIBLE_DEVICES`:

```
ssh <netid>@mlds-deepdish3.ads.northwestern.edu       # campus network or the VPN
cd RL && source .venv/bin/activate
CUDA_VISIBLE_DEVICES=0 python run.py pg --part 1 --env pong
```

For long runs, `scripts/jobs.sh` starts the command in the background on one card, so it keeps running after you log out:

```
# arguments: job name, GPU index, command
scripts/jobs.sh start part1_pong 0 python run.py pg --part 1 --env pong
scripts/jobs.sh status                    # state, exit code and GPU of every job
scripts/jobs.sh tail part1_pong 50        # last 50 lines of the job's log
scripts/jobs.sh stop part1_pong
```

7. Rebuilding the report

```
python run.py report
```

writes `report/report.md` and `report/report.pdf` (the report) and `report/instructions.pdf` (these instructions) from the saved answers, plots, evaluation statistics and test results. The **Overview** tab of the dashboard does the same.

8. Repository layout

Path	Contents
`run.py`	entry point for every command above
`rl/dp/`	Section 2: the shuttle-dispatch MDP, enumeration, value iteration and policy iteration
`rl/pg/`	policy gradient: environments, policy network, training and evaluation, baselines
`rl/cuda/`	CUDA kernels (Numba) for Section 2 and the policy-gradient arithmetic, with a CPU simulator fallback
`rl/plotting.py`	the plots the handout requires
`tests/`	the validation experiments and the harness that runs them; `tests/unit/` the unit tests
`lean/`	Lean 4 proofs about the algorithms and the exact reference values they produce
`results/dp/`, `results/pg/<run>/`, `results/tests/`	Section 2 results, one folder per policy-gradient run, validation results
`report/`	the report (`report.pdf`, `report.md`), the saved answers, these instructions and the report builder
`ui/`	the dashboard
`scripts/jobs.sh`	background job runner, one GPU card per job

2. Value and Policy Iteration (30 points)

Solution overview

> Provide a description of your solution and any corresponding plots.

Thinking from a greedy algorithm perspective, generally speaking we know that it is not worth it for our shuttle to move to the other platform if the cost of moving the shuttle is more than us waiting until that number breaks even. The state is the amount of people waiting at Station A, the action is taking people on the shuttle to station B. Reward is revenue-cost, and random demand is the amount of people arriving at the station.

In order to map this out, we should try with, explicitly we have 3 different methods we will explore in this Value and Policy iteration section: Assume the number of people can't exceed 200, and use discount rate of 0.95 (and additional assumptions will be listed too)

1. Enumeration: backward induction, so we're assuming a Time period = 500, we start at the last Time step, and apply the bellman backup going backwards
2. Value iteration: Repeat Bellman on one value table until it converges then read the best policy off the table, time = infinite.
3. Policy iteration: evaluate and greedy change policy and stop when policy stops changing

For optimal implementation, I have also asked claude to generate the cuda files which will generate the grids on the cores and be able to generate faster results with its instructions compared to python

MDP formulation

> Model the shuttle dispatching problem as a Markov Decision Process, given K = 15, c_f = 100, c_h = 2 and A_t ~ Unif{1, 2, 3, 4, 5}. The number of people in the station cannot exceed 200; use discount rate γ = 0.95.

Markov Decision Process means 5 things we should define: 1. state 2. action 3. transition 4. cost and 5. objective Reward is revenue-cost, and random demand is the amount of people arriving at the station.

1. State: The state is the amount of people that are currently on the platform A
2. Action: the action is taking people on the shuttle to station B
3. Transition: 3a. Number of people left on Station A after we transport is max(0, current_size - K). # K is capacity of a shuttle if it is dispatched 3b. If we don't dispatch shuttle, the amount of people would be current_size, 3c. at each timestep its's randomly going to be Unif {1, 2, 3, 4, 5}, so next state is = to min(people left + A_t, 200).
4. Cost: we have two costs in this problem: 1. Dispatching the shuttle will cost 100 per time step, and cost per customer on platform is 2c per timestep incurred after dispatch, with c being customer.
5. Our objective function is maximizing our reward, which is defined by the revenue - cost, or in this case because we don't have revenue, we just want to minimize our cost over time: min(cost)

(a) Enumeration, T = 500

> Solve using enumeration (with time period T = 500). Plot the optimal value function at time 0 versus number of customers waiting.

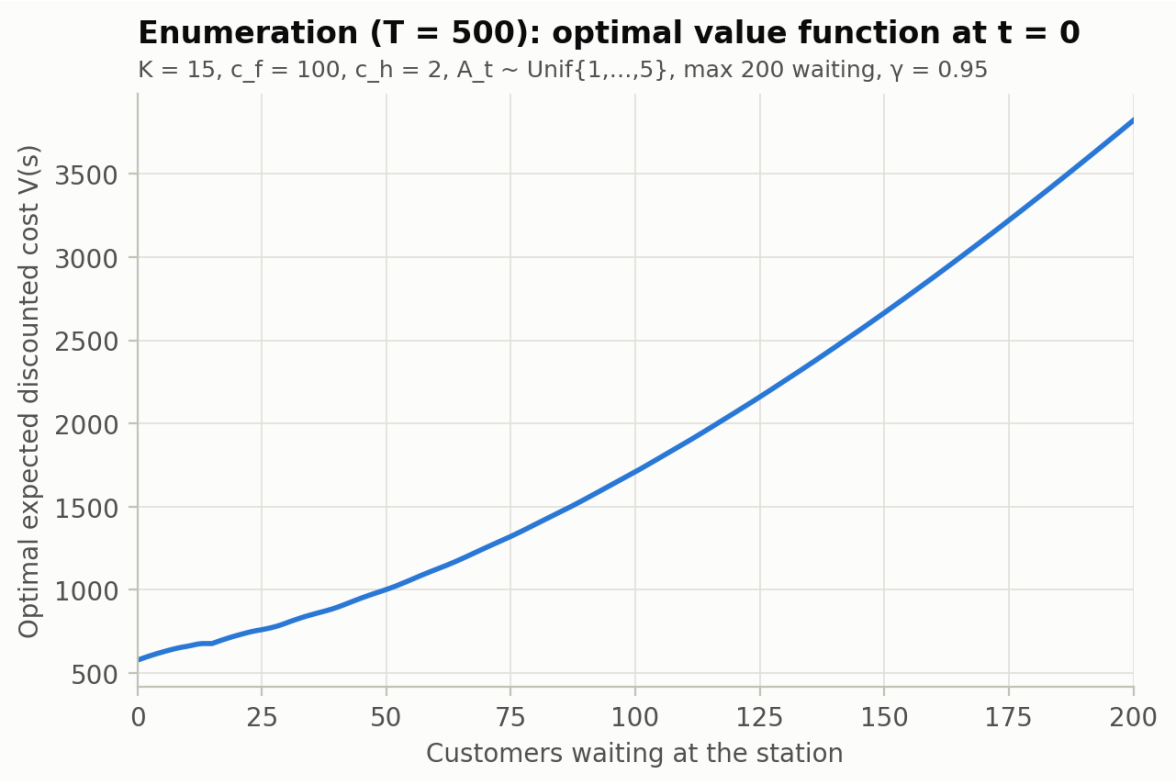


Figure 1. Section 2 (a), enumeration with T = 500: optimal value function at time 0 versus number of customers waiting.

In our very last case assume that it will not cost us anything at the last step. Technically speaking because the timestep is so far into the future, making this assumption is fine, that being said its possible to do more work in the future in this regards.

We performed backwards induction (essentially (dynamic programming) with time = 500 we apply the bellman backup and record the minimizing cost. Within this plot it shows that Essentially by computing it backwards, we know that at timestep for timestep we're computing a table of all the possible people in the station A. On the x axis we show the amount of people waiting at Station A when we end at our timestep of 0 (remember as we're starting from 500 and going to 0). Our Y axis shows our expected value at our last timestep.

(b) Value iteration, T = ∞

> Solve using value iteration (T = ∞). Plot optimal value function versus number of customers waiting.

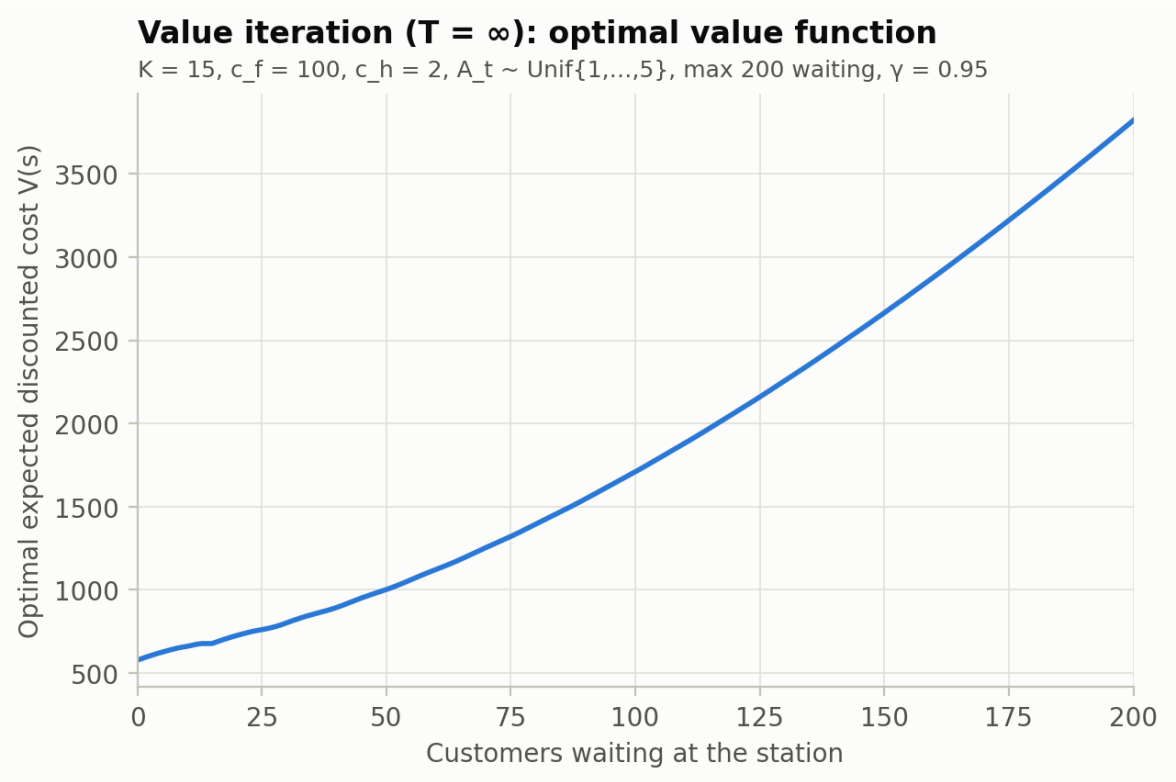


Figure 2. Section 2 (b), value iteration (T = ∞): optimal value function versus number of customers waiting.

We start with V(s) = 0 and then for every s value, we apply the Bellman backup and repeat. And thus our convergence criteria is when largest V change across all states is below our condition (which in our case we set it to 10^-6). Due to our discount factor by 0.95, our errors keep shrinking . Value iteration has no timesteps (in comaprison to enumeration) and it's similar to enumeration, however we don't work from our 500 timestep value backwards. We assume to have some sort of convergence criteria, which we'll define to 10^-6, and the discount factor allows us to compare future costs with todays' costs. It converged in 340 steps in comparison to before for our enumeration (since we started at T=500), due to our convergence factor.

(c) Policy iteration

> Solve using policy iteration. Plot optimal policy versus number of customers waiting.

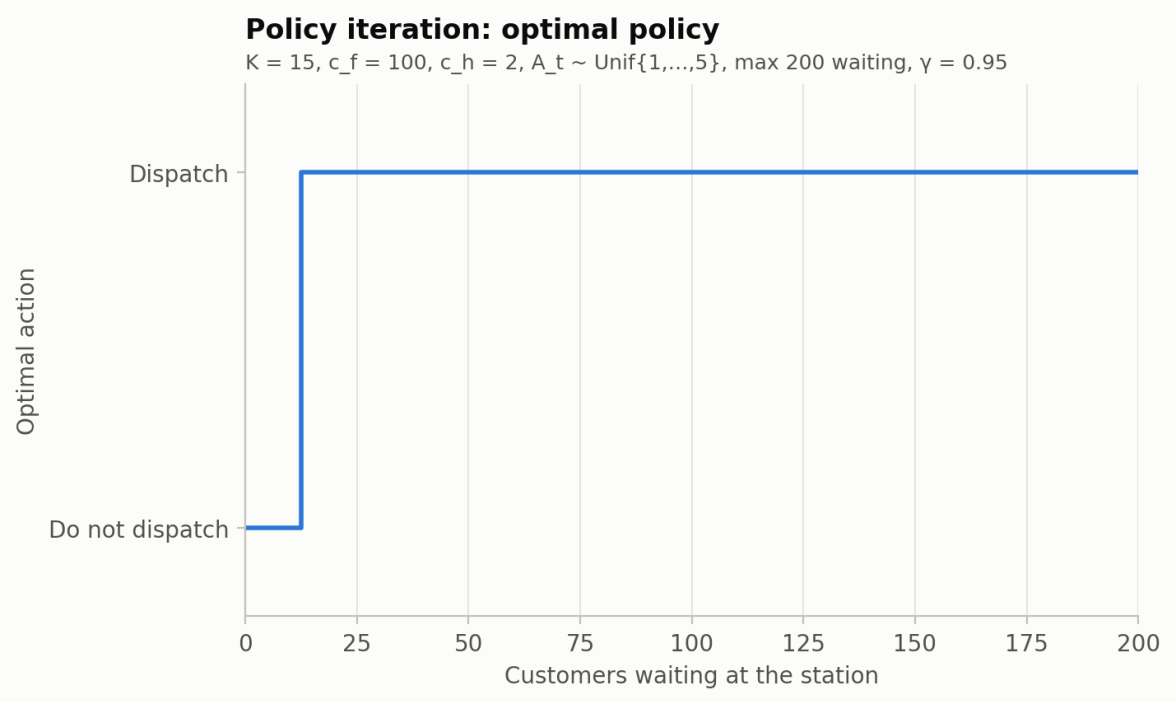


Figure 3. Section 2 (c), policy iteration: optimal policy versus number of customers waiting.

In our Policy iteration, we. have two actions: either we dispatch, or we hold in our station A. Policy iteration is also another form of DP problems. When we're looking at every customer s step, essentially because we're keeping track of all possible actions at every customer step we will always be able to solve the optimal strategy at each customer step. Thus essentially we're stepping through all possible 200 states (assume the empty platform s = 0) would be 201 states. At each state we evaluate two actions: whether we should dispatch or if we should hold. As we can see at the graph above, it makes sense for us to always dispatch when our value >= 13, to create our optimal policy.

The reason why this is called policy iteration is we then take this information and run it through

Assumptions

> If you make any other assumption, please state it in your report.

The number of people can't exceed 200, and Discount rate of 0.95

Edge cases

> What edge cases may arise from the problem and how did you handle them? Document the edge cases of the problem that you considered and explain how you ensured that your solution has handled them correctly.

Edge case 1: Exceeding the number of people on the platform. What would happen if we exceed the number of people we can hold on Platform A, aka if the amount of passengers goes above 200 due to an exponential amount of people coming into the station?

Edge case 2: There's a lot in this prompt that we can technically break, for example: Unif {1, 2, 3, 4, 5}, assuming a world where, we can think of an example lets say at an arbitrary timestep

we have 4 people come in, next time step we have another 4 come in, and then another 4, then we only get 5 in for the next 3 timesteps. What should the optimal policy be in this case? until when do we hold all of the people.

Edge case 3: Wince it's not possible for us to have 0 or - amount of people that part of the edge case is fine. What if we assume that the original platform has 200 people? and we have 5 people arriving at every timestep. So every number above 200 will be discarded assumingly since we can't go above 200 people. Would it make more sense for us to not dispatch to reduce the amount of cost? would that ever be the case.

Tests for validating the correctness of your algorithm

> Beyond asking AI, how do you know that your algorithm is working correctly? Test whether your implementation is behaving as predicted. For example, you may change certain parameters or the structure of the problem and see if the output matches your expectation. You may also reduce the stochasticity of the environment or policy so that the process is close to deterministic. You may also find it useful to isolate the first 10 steps and observe how the states, values, rewards, and policy interact at each step. Provide directions and documentation for how to execute validation and test experiments (what does each test do, how does it validate, etc). Provide quantitative and convincing evidence. (Not unit tests or basic bug fixes, such as checking the data format.)

Validation results from `python run.py tests --section dp` (last run 2026-10-26 12:34:38 on mlds-deepdish2):

Test	What it does	How it validates	Result
Three solvers, one answer	Solves the handout's problem (K = 15, c_f = 100, c_h = 2, A ~ Unif{1..5}, at most 200 waiting, γ = 0.95) with enumeration over T = 500 periods, value iteration and policy iteration, and compares the three value functions and policies state by state.	The three methods are different algorithms for the same optimality equation, so they must agree: enumeration with T = 500 differs from the infinite-horizon solution by at most γ^500 · max V ≈ 3e-8, and value iteration's stopping rule guarantees ‖V − V*‖ ≤ γ·tol/(1 − γ) (Lean: vi_error_bound). The values of policy iteration must have a zero Bellman residual, all policies must coincide, and the optimal policy must be a threshold rule (dispatch once enough customers wait).	PASSED (6/6 checks)

Test	What it does	How it validates	Result
Value iteration contracts at rate γ	Runs value iteration for γ ∈ {0.5, 0.8, 0.95} and records the change ‖V_{n+1} − V_n‖∞ after every iteration, the ratio of successive changes, and the distance of the final iterate from the policy-iteration solution.	The Bellman operator is a γ-contraction (Lean: bellman_contraction), so successive changes must shrink by at least the factor γ every iteration (value_succ_sub_bound) and the number of iterations to a tolerance must scale like log(tol)/log(γ). The stopping rule's a-posteriori bound γ·Δ/(1 − γ) must dominate the true error (vi_error_bound). Ratios are only examined while the changes are far above floating-point noise.	PASSED (12/12 checks)
The first 10 steps of backward induction, and 10 periods of the optimal policy	On a station capped at 30 customers (same costs, K and arrivals as the handout), records the value function and the optimal first action after each of the first 10 backward-induction steps, recomputes the first two steps by hand in exact rational arithmetic, and then follows the optimal policy for 10 periods from an empty station, printing state, action, cost and next state.	With one period left no shuttle is worth dispatching ($100 + 2\cdot(s-15)^+ > 2s$ for $s \le 30$), so $V_1(s) = 2s$ exactly; V_2 must equal the hand computation; values must grow with the horizon (Lean: value_mono_horizon) and with the queue (value_mono_state); the dispatch threshold must appear and settle as more periods are added. The 10-period trajectory lets one verify each cost and transition by hand (dispatch exactly when the queue reaches the threshold, 15 customers leave, 1-5 arrive).	PASSED (7/7 checks)
Exact rational arithmetic agrees with the floating-point solvers	Re-solves four small instances (two arrival values; a non-uniform arrival distribution; deterministic arrivals; the handout's costs on a 20-customer station) by backward induction in exact rational arithmetic, twice: with Python fractions and with the Lean definition that the formal proofs are about (lean/RLVerify/Shuttle.lean, exported to lean/reference/shuttle_reference.json), and compares with enumeration.	The floating-point solver must reproduce the exact values to round-off (1e-12) at every horizon and state, and the optimal first actions must coincide. The Lean values are computed by the very definition whose properties are proved, so this ties the Python implementation to the verified model.	PASSED (3/3 checks)
γ → 0 makes the agent myopic; more foresight dispatches earlier	Solves the problem for γ from 0 to 0.99 and records the dispatch threshold and V(0).	With γ = 0 only the current period counts, and dispatching (≥ 100) is never cheaper than holding (≤ $2\cdot200$ but $100 + 2(s-15) > 2s$), so the myopic agent never dispatches and $V(s) = 2s$ exactly. As γ grows, future holding costs weigh more, so the threshold must not increase and V(0) must increase.	PASSED (3/3 checks)

Test	What it does	How it validates	Result
Cost parameters move the dispatch threshold the right way	Sweeps the dispatch cost c_f, the holding cost c_h and the capacity K one at a time and records the optimal dispatch threshold; also solves the degenerate cases c_f = 0, c_h = 0 and K = 0.	A dearer shuttle must not be dispatched earlier (threshold non-decreasing in c_f); dearer waiting must not delay dispatch (threshold non-increasing in c_h). Free shuttles are dispatched as soon as anyone waits, free waiting or a useless (K = 0) shuttle means never dispatching.	PASSED (5/5 checks)
Changing the structure of the problem	Solves variants of the model: (a) up to two shuttles per period; (b) holding cost charged at the end of the period instead of right after the dispatch; (c) holding charged before the dispatch; (d) a station cap of 400 instead of 200; (e) deterministic arrivals (always 3).	(a) More actions can only lower the optimal cost, and the second shuttle is only used for long queues. (b) Charging c_h·(left + A) instead of c_h·left adds the constant c_h·E[A] per period, so the policy must be identical and every value must shift by exactly c_h·E[A]/(1 − γ) = 120. (c) Charging everyone present costs more, so values rise. (d) Under the optimal policy the queue never approaches 200, so the cap is irrelevant: values must agree to round-off. (e) With deterministic arrivals a single simulated trajectory has no noise, so its discounted cost must equal V(0) to within γ^horizon.	PASSED (9/9 checks)
Monte Carlo rollouts reproduce the value function	Simulates the optimal policy from several starting queues (5000 trajectories of 600 periods each) and compares the average discounted cost with V*(s); also simulates two deliberately worse threshold policies (dispatch at 5 and at 30).	V*(s) is defined as the expected discounted cost of the optimal policy, so the sample mean must land within three standard errors (plus the negligible truncation error γ^600·max cost/(1 − γ)). Any other policy must cost at least as much in every state, both in simulation and when evaluated exactly.	PASSED (8/8 checks)
Policy iteration improves monotonically and ends at the greedy fixed point	Records every policy visited by policy iteration (starting from 'never dispatch' and, separately, from 'always dispatch'), evaluates each exactly and compares consecutive value functions.	The policy-improvement theorem (Lean: policy_improvement) says the greedy policy for V^π is at least as good as π in every state, so the sequence of value functions must be pointwise non-increasing, strictly better somewhere at each step, and both starting points must reach the same policy, which must be greedy with respect to its own values.	PASSED (3/3 checks)

Test	What it does	How it validates	Result
The CUDA kernels reproduce the solvers	Runs the GPU versions of Section 2 (rl/cuda/dp.py: the state grid initialised on the device, one thread per state for the Bellman sweeps with the value function in shared memory, greedy improvement, and one thread per trajectory for Monte Carlo) and compares them with the numpy solvers. On a machine with an NVIDIA GPU this is the full 201-state problem and 100000 simulated trajectories; without one, the same kernels run in Numba's CUDA simulator on a 31-state instance (slow, so the sizes are small).	The kernels implement the same equations with a different execution order (per-thread sparse sums, block-wide max reductions), so values must agree to round-off, policies and iteration counts must coincide, and the per-trajectory simulation must land within three standard errors of V*. The recorded kernel times are real only on a GPU.	PASSED (6/6 checks)

- **Three solvers, one answer**: max |V_enum − V_PI| = 4.94e-09, max |V_VI − V_PI| = 1.875e-08, max |V_enum − V_VI| = 1.381e-08, VI iterations = 474, VI error bound = 1.876e-08, PI improvement steps = 6, dispatch threshold = 13, V*(0) = 576.7, V*(200) = 3822
- **Value iteration contracts at rate γ**: iterations (γ = 0.5) = 37, iterations (γ = 0.8) = 101, iterations (γ = 0.95) = 429
- **Exact rational arithmetic agrees with the floating-point solvers**: max |float − exact fractions| = 2.842e-14, max |float − Lean exact| = 2.842e-14
- **γ → 0 makes the agent myopic; more foresight dispatches earlier**: threshold at γ = 0.95 = 13
- **Cost parameters move the dispatch threshold the right way**: threshold range over the c_f sweep = 4–1000000000
- **Changing the structure of the problem**: (a) largest saving from a second shuttle = 1011, (c) threshold when holding is charged before dispatch = 13
- **Policy iteration improves monotonically and ends at the greedy fixed point**: improvement steps from 'never dispatch' = 6, improvement steps from 'always dispatch' = 5
- **The CUDA kernels reproduce the solvers**: enumeration kernel time (s) = 0.4445, value iteration kernel time (s) = 0.0074, policy iteration kernel time (s) = 0.2702, simulation kernel time (s) = 1.155, simulation grid (blocks × threads) = 391 × 256

We imported the lean module to verify the mathematical correctness for the program. Additionally in explore the problem we manually stepped through each of the states for 10 states to evaluate if the program is working as intended

Additional validation with 3 more months

> Given an additional 3 months (extended period), what other validation tests and experiments would you perform?

- One thing I would like to do is for enumeration for us to not assume the condition that we will not incur any costs at our timestep of 500. Assuming the case where there is a terminal valye of 1 trillion for every s, then we get very super marginal changes in our value at

timestep 0 (i.e. $7.27)

- We can start breaking a lot of hte original prompt by assuming a few things: i.e. if we start having platform B also have people moving to Platform A, we're assuming there's an amount of people in the car existing, and it doesn't fully clear all the time at B.

3. Policy Gradient: Part 1 (40 points)

Solution overview

> Using PyTorch, implement the basic policy gradient algorithm: train a simple neural network that models the policy and takes actions based on the provided states. Set the discounted reward over an episode as your objective function and use Adam as your gradient optimizer with learning rate 0.01 (if you use a different learning rate because training is more stable, mention it). Provide a description of your solution and any corresponding plots.

Answer pending.

(a) CartPole-v1, discount factor 0.95

> Plot the episode reward against the number of training episodes, overlaid with the moving average of the rewards over 100 episodes. When you finish training, roll out 500 episodes using your trained model. Plot the histogram of the 500 episode rewards and report the mean and the standard deviation.

Figure 4 (pending). Part 1 (a), CartPole-v1: episode reward versus training episode, with the 100-episode moving average. Generate it with `python run.py pg --part 1 --env cartpole`*.*

Figure 5 (pending). Part 1 (a), CartPole-v1: histogram of the rewards of 500 episodes rolled out with the trained policy. Generate it with `python run.py pg --part 1 --env cartpole`*.*

Evaluation over 500 episodes: mean and standard deviation pending.

Answer pending.

(b) Pong-v5, discount factor 0.99

> Use frames of the game as input to a neural net which models the policy. The game has six actions [NOOP, FIRE, RIGHT, LEFT, RIGHTFIRE, LEFTFIRE], but you should train a network that chooses the best action only between [RIGHT, LEFT] (actions 2 and 3). You may pre-process the frames with the provided preprocess function. Use the specified environment version (Pong-v5); on deepdish, use a single GPU card. Plot the episode reward against the number of training episodes, overlaid with the moving average of the rewards over 100 episodes. When you finish training, roll out 500 episodes using your trained model. Plot the histogram of the 500 episode rewards and report the mean and the standard deviation.

Figure 6 (pending). Part 1 (b), Pong-v5: episode reward versus training episode, with the 100-episode moving average. Generate it with `python run.py pg --part 1 --env pong`*.*

Figure 7 (pending). Part 1 (b), Pong-v5: histogram of the rewards of 500 episodes rolled out with the trained policy. Generate it with `python run.py pg --part 1 --env pong`.

Evaluation over 500 episodes: mean and standard deviation pending.

Answer pending.

Edge cases

> What edge cases may arise from the problem and how did you handle them? Document the edge cases of the problem that you considered and explain how you ensured that your solution has handled them correctly.

Answer pending.

Tests for validating the correctness of your algorithm

> Beyond asking AI, how do you know that your algorithm is working correctly? Test whether your implementation is behaving as predicted. For example, you may change certain parameters or the structure of the problem and see if the output matches your expectation. You may also reduce the stochasticity of the environment or policy so that the process is close to deterministic. You may also find it useful to isolate the first 10 steps and observe how the states, values, rewards, and policy interact at each step. Provide directions and documentation for how to execute validation and test experiments (what does each test do, how does it validate, etc). Provide quantitative and convincing evidence. (Not unit tests or basic bug fixes, such as checking the data format.)

Validation results from `python run.py tests --section pg1` (last run 2026-10-26 12:35:40 on mlds-deepdish2):

Test	What it does	How it validates	Result
The implemented gradient equals the analytic policy gradient	For a linear softmax policy, compares the gradient PyTorch computes from the surrogate loss with the closed form $-\Sigma_t G_t (e_{a_t} - \pi(\cdot\|s_t)) \otimes s_t$, on single-step bandit episodes with random logits and on multi-step corridor episodes.	REINFORCE's update is $\Sigma_t \nabla \log \pi(a_t\|s_t)\, G_t$ and the score of a softmax is $e_a - \pi$, so the two must agree to floating-point precision for every action and return; a sign error, a missing return, or a wrong time alignment would show up immediately.	PASSED (2/2 checks)

Test	What it does	How it validates	Result
Bandits: the policy concentrates on the better arm	Trains the policy on one-step problems: a deterministic two-armed bandit with rewards −1 and +1, a Bernoulli bandit with success probabilities 0.3 and 0.7, and a three-armed bandit with positive rewards 0, 1 and 0.5 (10 seeds).	Without state or sequence effects, REINFORCE is plain stochastic gradient ascent on E[r], so the probability of the best arm must approach 1 in the first two problems. The third problem documents a known weakness that motivates Part 2: with positive-only rewards and no baseline, every sampled arm is reinforced, so some seeds lock onto the 0.5 arm sampled early; the fraction that still finds the best arm is recorded (and compared with baselines in the Part 2 experiments).	PASSED (2/2 checks)
REINFORCE reaches the optimum computed by value iteration (corridor)	A corridor of 6 cells with the reward at the end (deterministic, and with a 20% chance that a move is reversed). The exact optimal policy and value come from the Section 2 solvers on the same MDP; REINFORCE is trained for 400 episodes and its greedy policy is evaluated exactly on the MDP.	Two independent implementations must agree: the greedy policy extracted from the trained network must be 'always right' (the DP optimum), its exact value must equal the DP optimal value, and in the deterministic case the discounted return of a greedy rollout must be γ^(L−2) exactly. The stochastic policy's own average return must be close to the optimum once the per-state probabilities of 'right' are near 1.	PASSED (7/7 checks)
The discount factor decides whether a delayed reward is worth pursuing	A 5-cell corridor where every move to the right costs 0.1 and the goal pays 1. For γ ∈ {0, 0.5, 0.9, 0.99} the Section 2 solvers give the exact optimal policy of the corresponding MDP (go right from the start or not), and REINFORCE is trained with the same γ (3 seeds, 300 episodes); the success rate at the end of training is compared with what the optimal policy does. Also checks the return weights of one episode.	With γ = 0 only the immediate reward counts, so moving right (−0.1) is never worth it and the optimal agent stays at the start; with γ = 0.9 the discounted goal reward outweighs the move costs. Value iteration says exactly which γ flip the decision, and REINFORCE trained with each γ must reproduce that decision: near-zero success where the optimum is to stay, near-certain success where it is to go. This isolates the discount factor from every other part of the algorithm.	PASSED (4/4 checks)

Test	What it does	How it validates	Result
Three ways to weight the log-probabilities: which agree in expectation	With a fixed random policy on a stochastic corridor that charges 0.05 per step (so rewards arrive at every step, not only at the goal), draws 600 single-episode gradient estimates for three weightings of ∇log π(a_t|s_t): the whole-episode return G_0 (classic REINFORCE), γ^t·G_t (the exact gradient of the discounted objective, `--set discount_grad=true`) and the reward-to-go G_t without the γ^t factor (the implementation's default, as in most practical code), and compares means and variances.	Rewards earned before step t are independent of a_t given the history, so dropping them does not change the expectation: the G_0 and γ^t·G_t estimates must have the same mean within sampling error, and here the γ^t·G_t weights are pointwise smaller, so its variance must be lower. The default G_t weighting drops the γ^t factor and therefore estimates a slightly different gradient (it treats every visited state as a start state); the experiment records that difference so the choice is a documented one.	PASSED (2/2 checks)
CartPole-v1 learns within 200 episodes on the CPU	Trains with the handout's settings (Adam, learning rate 0.01, γ = 0.95) for 300 episodes, seed 0, then evaluates 50 episodes; records the mean reward of the first 20 and of the last 50 training episodes.	A correct REINFORCE implementation improves CartPole quickly: the last 50 episodes must average at least 1.5 times the first 20 and at least 100 steps, and the evaluation mean must exceed 50 (random play lasts about 22). The full 1000-episode run and its plots are produced by `python run.py pg --part 1 --env cartpole`.	PASSED (2/2 checks)
Pong-v5 pipeline: preprocessing and the two-action restriction	Plays one full game of ALE/Pong-v5 with an untrained policy, recording every action sent to the environment and statistics of the preprocessed frame differences the network sees.	The handout restricts the agent to RIGHT (2) and LEFT (3): every recorded action must be one of them. The preprocessed input must be an 80×80 image with values in {−1, 0, 1} (frame difference of binary frames) that is non-zero most of the time (the ball and paddles move), and a game must end with a score between −21 and 21 (an untrained agent loses almost every point).	PASSED (5/5 checks)

- **The implemented gradient equals the analytic policy gradient**: max |autograd − analytic| (multi-step) = 8.277e-08
- **Bandits: the policy concentrates on the better arm**: P(best arm), rewards ±1 = 1, P(best arm), Bernoulli 0.3/0.7 = 0.9998, positive-only rewards: seeds finding the best arm (of 10) = 6
- **REINFORCE reaches the optimum computed by value iteration (corridor)**: optimal value (slip 0.0) = 0.6561, optimal value (slip 0.2) = 0.5132
- **The discount factor decides whether a delayed reward is worth pursuing**: success rate (γ = 0.0) = 0, success rate (γ = 0.5) = 0.3333, success rate (γ = 0.9) = 1, success rate (γ = 0.99) = 1
- **Three ways to weight the log-probabilities: which agree in expectation**: max |z|: G_0 vs γ^t G_t = 0.415, max |z|: G_0 vs G_t = 7.645, variance ratio γ^t G_t / G_0 = 0.5559

- **CartPole-v1 learns within 200 episodes on the CPU**: mean reward, episodes 1-20 = 43.2, mean reward, episodes 251-300 = 206.1, evaluation mean (50 episodes) = 360.9, evaluation std = 44.99, training time (s) = 18.3
- **Pong-v5 pipeline: preprocessing and the two-action restriction**: steps in the game = 921, game score (reward sum) = -20, fraction of frame differences with any non-zero pixel = 0.9881, seconds per step (CPU, untrained MLP) = 0.001

Answer pending.

Additional validation with 3 more months

> Given an additional 3 months (extended period), what other validation tests and experiments would you perform?

Answer pending.

Policy Gradient: Part 2 (20 points)

Baseline: choice and implementation

> The training suffers from high variance of the rewards in the sampled episodes; using a baseline helps to reduce the variance. Repeat the experiments of Part 1 using a constant or non-constant baseline of your choice. Explain which baseline you have chosen and how you implemented it.

Answer pending.

Solution overview

> Provide a description of your solution and any corresponding plots.

Answer pending.

(a) CartPole-v1 with baseline

> Repeat Part 1 (a) with your baseline. Plot the episode reward against the number of training episodes, overlaid with the moving average of the rewards over 100 episodes. When you finish training, roll out 500 episodes using your trained model. Plot the histogram of the 500 episode rewards and report the mean and the standard deviation.

Figure 8 (pending). Part 2 (a), CartPole-v1: episode reward versus training episode, with the 100-episode moving average. Generate it with `python run.py pg --part 2 --env cartpole`*.*

Figure 9 (pending). Part 2 (a), CartPole-v1: histogram of the rewards of 500 episodes rolled out with the trained policy. Generate it with `python run.py pg --part 2 --env cartpole`*.*

Evaluation over 500 episodes: mean and standard deviation pending.

Answer pending.

(b) Pong-v5 with baseline

> Repeat Part 1 (b) with your baseline. Plot the episode reward against the number of training episodes, overlaid with the moving average of the rewards over 100 episodes. When you finish training, roll out 500 episodes using your trained model. Plot the histogram of the 500 episode rewards and report the mean and the standard deviation.

Figure 10 (pending). Part 2 (b), Pong-v5: episode reward versus training episode, with the 100-episode moving average. Generate it with `python run.py pg --part 2 --env pong`.

Figure 11 (pending). Part 2 (b), Pong-v5: histogram of the rewards of 500 episodes rolled out with the trained policy. Generate it with `python run.py pg --part 2 --env pong`.

Evaluation over 500 episodes: mean and standard deviation pending.

Answer pending.

Edge cases

> What edge cases may arise from the problem and how did you handle them? Document the edge cases of the problem that you considered and explain how you ensured that your solution has handled them correctly.

Answer pending.

Tests for validating the correctness of your algorithm

> Beyond asking AI, how do you know that your algorithm is working correctly? Test whether your implementation is behaving as predicted. For example, you may change certain parameters or the structure of the problem and see if the output matches your expectation. You may also reduce the stochasticity of the environment or policy so that the process is close to deterministic. You may also find it useful to isolate the first 10 steps and observe how the states, values, rewards, and policy interact at each step. Provide directions and documentation for how to execute validation and test experiments (what does each test do, how does it validate, etc). Provide quantitative and convincing evidence. (Not unit tests or basic bug fixes, such as checking the data format.)

Validation results from `python run.py tests --section pg2` (last run 2026-10-26 12:37:04 on mlds-deepdish2):

Test	What it does	How it validates	Result
A baseline leaves the expected gradient unchanged and changes its variance	On a Bernoulli bandit (success probabilities 0.3 and 0.7) with a fixed policy, draws 2000 single-episode gradient estimates with no baseline, with the constant baseline b = 0.5 (the mean reward), with a deliberately bad constant b = 5, and with an untrained value network, and compares their means and variances.	Any baseline that does not depend on the action keeps the estimator unbiased (Lean: baseline_unbiased), so all four means must agree within sampling error. The variance is minimised near the mean return: b = 0.5 must have lower variance than no baseline, and b = 5 a much higher one, which is why the baseline has to be fitted rather than chosen arbitrarily.	PASSED (5/5 checks)

Test	What it does	How it validates	Result
Baselines prevent lock-in on the first rewarded arm	Three-armed bandit with positive rewards 0, 1 and 0.5, learning rate 0.03, 300 episodes, 10 seeds, trained without a baseline and with the mean, time and value baselines; counts the seeds whose final policy prefers the best arm.	Without a baseline every sampled arm is reinforced (all returns are positive), so Adam can lock onto the 0.5 arm before the 1.0 arm is ever tried. A baseline near the average return makes the 0.5 arm's advantage zero or negative, so the policy keeps exploring until it finds the 1.0 arm: the baselines must find the best arm at least as often as plain REINFORCE.	PASSED (3/3 checks)
The value baseline learns the policy's state values	Trains REINFORCE with the value baseline on a 6-cell corridor (10% slip, $\gamma = 0.9$) for 500 episodes, then compares the value network's prediction in every state with the exact value of the learned stochastic policy (computed by policy evaluation on the corridor MDP) and with the optimal value.	The baseline network is fitted by least squares to the observed returns, whose expectation is the value of the current policy, so $V_\varphi(s)$ must be close to $V^\pi(s)$ in the states the policy visits (within 0.1 on a 0-1 scale) and must increase towards the goal. Because the policy is near-optimal by the end, V_φ is also close to V^*. This checks that the baseline is state-dependent and informative, not a constant.	PASSED (3/3 checks)
Learning curves with and without a baseline	A 10-cell corridor with 15% slip, learning rate 0.02, 300 episodes, 5 seeds each for no baseline, the mean baseline and the value baseline; records the average success rate over training (area under the learning curve), the final success rate, and the root-mean-square of the per-step weights ($G_t - b_t$) relative to that of G_t, measured on fresh episodes with the final policy and baseline.	The variance of the gradient estimate scales with $E[(G_t - b_t)^2]$, so a useful baseline must shrink the root-mean-square weight (a constant baseline cannot change the standard deviation of the weights, only their mean square). Whether lower-variance updates also learn faster depends on the problem, so the areas under the learning curves are recorded and compared rather than asserted: on this sparse-reward corridor the only reward is a success, so plain REINFORCE's habit of reinforcing every success is not a handicap, while the value baseline first has to learn the values it subtracts.	PASSED (2/2 checks)
CartPole-v1 with the value baseline (200 episodes on the CPU)	Trains CartPole-v1 for 200 episodes with the value baseline (handout settings otherwise), evaluates 50 episodes, and measures on 20 fresh episodes how much the baseline shrinks the update weights (root-mean-square of $G_t - V(s_t)$ relative to G_t).	The baseline must not stop CartPole from learning (moving average at least doubles, evaluation mean > 50) and must shrink the root-mean-square weight, which governs the variance of the gradient estimate, by a clear margin. The full 1000-episode run is `python run.py pg --part 2 --env cartpole`.	PASSED (3/3 checks)

- **A baseline leaves the expected gradient unchanged and changes its variance**: variance ratio (b = 0.5) / none = 0.5266, variance ratio (b = 5) / none = 51.23
- **Baselines prevent lock-in on the first rewarded arm**: seeds finding the best arm: none = 6, seeds finding the best arm: mean = 7, seeds finding the best arm: time = 8, seeds finding the best arm: value = 8
- **The value baseline learns the policy's state values**: max $|V_\varphi - V^\pi|$ over non-terminal states = 0.04492, max $|V_\varphi - V^*|$ = 0.04486

- **Learning curves with and without a baseline**: area under the learning curve: none = 0.9987, area under the learning curve: mean = 0.9987, area under the learning curve: value = 0.9687
- **CartPole-v1 with the value baseline (200 episodes on the CPU)**: moving average, episodes 1-50 = 53.4, moving average, episodes 151-200 = 209.7, evaluation mean (50 episodes) = 500, rms(G − V(s)) / rms(G) on fresh episodes = 0.1249, final value loss = 5.534

Answer pending.

Additional validation with 3 more months

> Given an additional 3 months (extended period), what other validation tests and experiments would you perform?

Answer pending.